

CS-1204
Object-Oriented Programming
(3+1)

Week 05

By

Engr. Muhammad Awais

Contents

- Type of Inheritance
- Diamond Problem
- Virtual Inheritance

~~_____~~

~~→ name, chit~~

E

n, ~~_____~~ S

~~_____~~ S

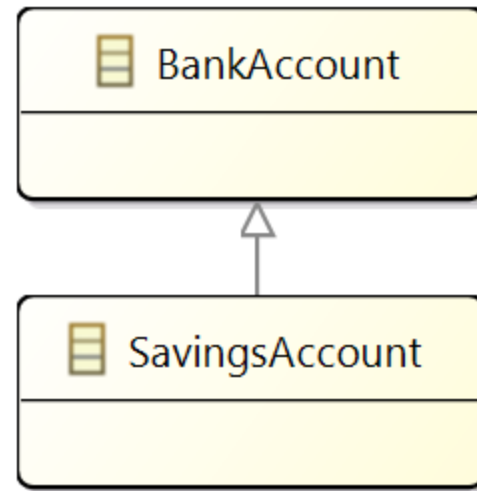
n, ~~_____~~ S

Types of Inheritance

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

Single Inheritance

- Single inheritance allows a **derived class to inherit from one base class.**
- It promotes code reuse by enabling a specialized class (e.g., SavingsAccount) to inherit and extend the behavior of a more general class (e.g., BankAccount).



Example Single Inheritance

// Base Class

```
class BankAccount {
private:
    string accountHolder;
    double balance;

public:
    BankAccount(string name, double initialBalance) {
        accountHolder = name;
        balance = initialBalance;
    }

    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    void withdraw(double amount) {
        if (amount > 0 && amount <= balance) balance -= amount;
    }

    double getBalance() {
        return balance;
    }

    string getAccountHolder() {
        return accountHolder;
    }
};
```

// Derived Class (Single Inheritance)

```
class SavingsAccount : public BankAccount {
private:
    double interestRate;

public:
    SavingsAccount(string name, double initialBalance, double rate)
        : BankAccount(name, initialBalance) {
        interestRate = rate;
    }

    void addInterest() {
        double interest = getBalance() * interestRate / 100;
        deposit(interest);
    }
};

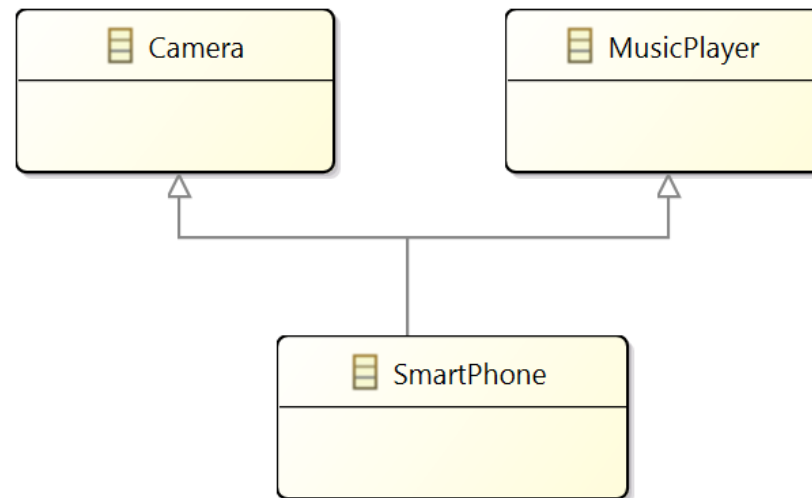
int main() {
    SavingsAccount acc("Ali", 10000, 5.0);
    acc.deposit(2000);
    acc.addInterest();
    acc.withdraw(1000);

    cout << "Account Holder: " << acc.getAccountHolder() << endl;
    cout << "Final Balance: " << acc.getBalance() << endl;

    return 0;
}
```

Multiple Inheritance

- Multiple inheritance enables **a class to inherit from more than one base class**, combining functionalities from multiple sources.
- For example, SmartPhone can inherit features of both a Camera and a MusicPlayer.



Example of Multiple Inheritance

```
// Camera class
class Camera {
public:
    void capturePhoto() {
        cout << "Photo captured!" << endl;
    }

    void recordVideo() {
        cout << "Video recording started!" << endl;
    }
};

// MusicPlayer class
class MusicPlayer {
public:
    void playMusic() {
        cout << "Music is playing!" << endl;
    }

    void pauseMusic() {
        cout << "Music paused!" << endl;
    }
};
```

```
// Smartphone class that inherits from Camera and MusicPlayer
class Smartphone : public Camera, public MusicPlayer {
public:
    void makeCall() {
        cout << "Making a call..." << endl;
    }

    void sendText() {
        cout << "Sending a text message..." << endl;
    }
};

int main() {
    Smartphone myPhone;

    // Using functionality from Camera
    myPhone.capturePhoto();
    myPhone.recordVideo();

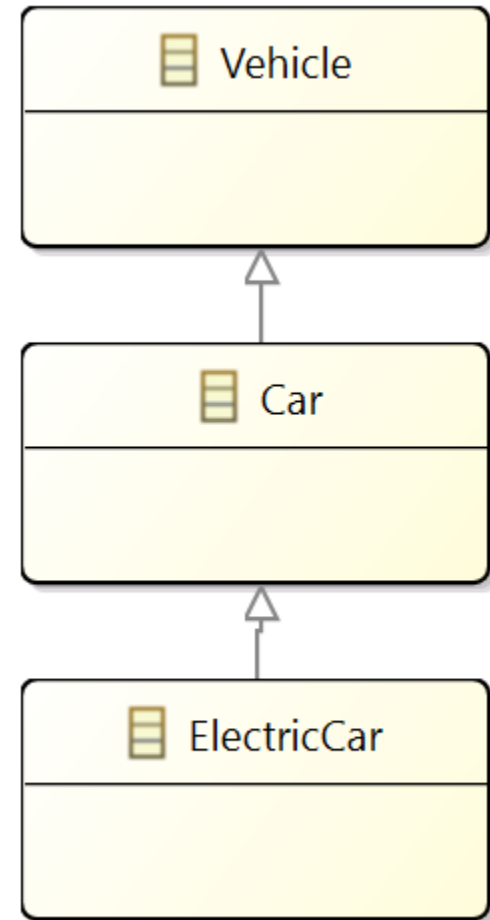
    // Using functionality from MusicPlayer
    myPhone.playMusic();
    myPhone.pauseMusic();

    // Using functionality specific to Smartphone
    myPhone.makeCall();
    myPhone.sendText();

    return 0;
}
```

Multilevel Inheritance

- Multilevel inheritance involves a **class derived from a class that is already derived from another class.**
- This forms a parent–child–grandchild hierarchy.
- ElectricCar inheriting features from both Car and Vehicle.



Example of Multilevel Inheritance

```
// Base class
class Vehicle {
public:
    void startEngine() {
        cout << "Engine started." << endl;
    }

    void stopEngine() {
        cout << "Engine stopped." << endl;
    }
};

// Derived class from Vehicle
class Car : public Vehicle {
public:
    void drive() {
        cout << "Car is being driven." << endl;
    }

    void honk() {
        cout << "Car horn is honking." << endl;
    }
};

// Derived class from Car (multilevel)
class ElectricCar : public Car {
public:
    void chargeBattery() {
        cout << "Battery is charging." << endl;
    }

    void showBatteryStatus() {
        cout << "Battery is at 85%." << endl;
    }
};
```

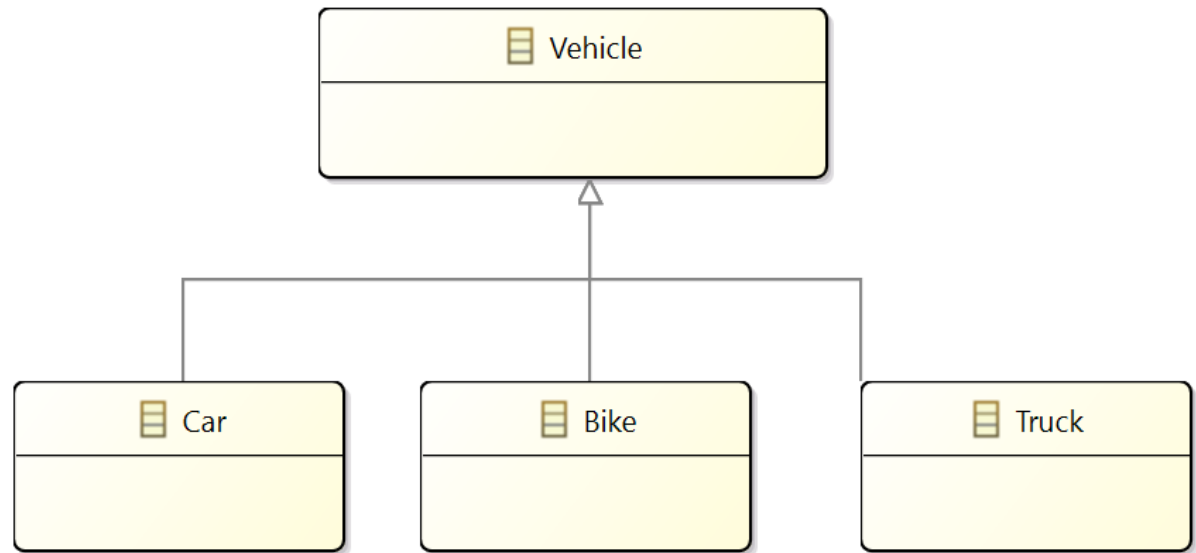
```
int main() {
    ElectricCar myTesla;

    // Accessing methods from all three levels
    myTesla.startEngine();    // from Vehicle
    myTesla.drive();          // from Car
    myTesla.chargeBattery();  // from ElectricCar
    myTesla.showBatteryStatus(); // from ElectricCar
    myTesla.honk();           // from Car
    myTesla.stopEngine();     // from Vehicle

    return 0;
}
```

Hierarchical Inheritance

- Hierarchical inheritance is when **multiple derived classes inherit from a single base class**.
- That means one parent class and many child classes.



Example of Hierarchical Inheritance

```
// Base class
class Vehicle {
public:
    void startEngine() {
        cout << "Engine started." << endl;
    }
};

// Derived class 1
class Car : public Vehicle {
public:
    void openTrunk() {
        cout << "Trunk is open." << endl;
    }
};

// Derived class 2
class Bike : public Vehicle {
public:
    void kickStart() {
        cout << "Bike kick started." << endl;
    }
};

// Derived class 3
class Truck : public Vehicle {
public:
    void loadCargo() {
        cout << "Cargo loaded into the truck." << endl;
    }
};
```

```
int main() {
    Car myCar;
    Bike myBike;
    Truck myTruck;

    // ALL have access to base class method
    myCar.startEngine();
    myCar.openTrunk();

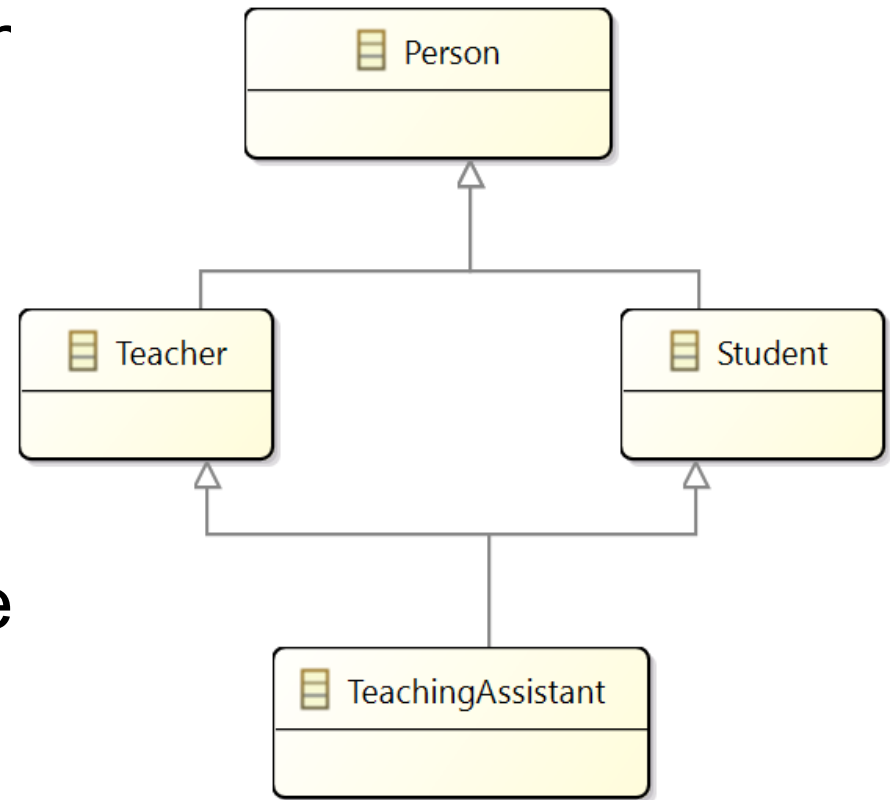
    myBike.startEngine();
    myBike.kickStart();

    myTruck.startEngine();
    myTruck.loadCargo();

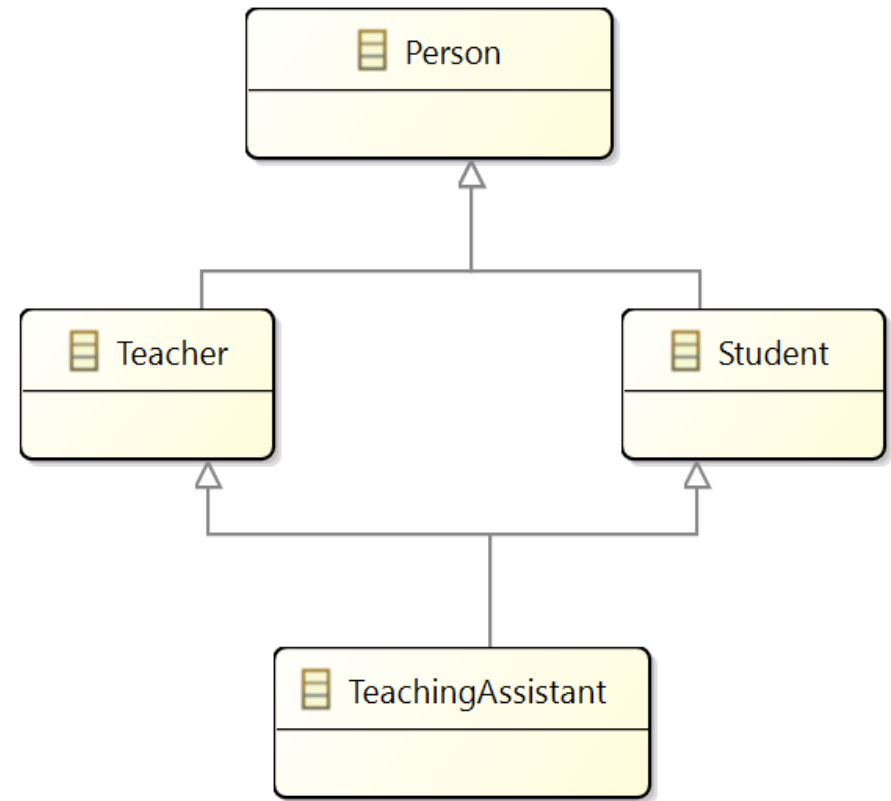
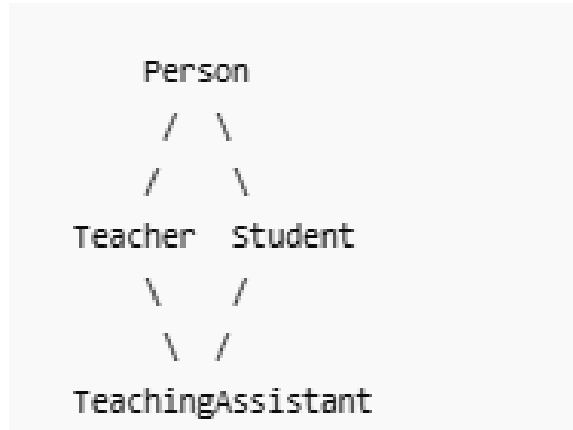
    return 0;
}
```

Hybrid Inheritance

- Hybrid inheritance is a combination of more than one type of inheritance, such as:
 - ❑ Multiple inheritance
 - ❑ Multilevel inheritance
 - ❑ Hierarchical inheritance
- Hybrid inheritance creates a more complex class relationship.



Diamond Problem



- TeachingAssistant has **multiple paths** to Person causing diamond problem.
- This makes **compiler confuse which path to use** when the object of TeachingAssistant class uses anything from class Person.
- This will generate a compile time error.

Example Code with Diamond Problem

```
#include <iostream>
using namespace std;

class Person {
public:
    void showIdentity() {
        cout << "I am a person." << endl;
    }
};

class Teacher : public Person {
public:
    void teach() {
        cout << "Teaching students." << endl;
    }
};

class Student : public Person {
public:
    void study() {
        cout << "Studying courses." << endl;
    }
};

class TeachingAssistant : public Teacher, public Student {
public:
    void assist() {
        cout << "Assisting in labs and grading." << endl;
    }
};
```

```
int main() {
    TeachingAssistant ta;
    ta.showIdentity(); // X ERROR: Ambiguous
    ta.teach();
    ta.study();
    ta.assist();
    return 0;
}
```

Error

Compiler (4) Resources Compile Log Debug Find Results Close			
Line	Col	File	Message
34	8	D:\Data\Awais_Laptop\University\Courses\Session\Sprin...	In function 'int main()':
6	10	D:\Data\Awais_Laptop\University\Courses\Session\Sprin...	[Error] request for member 'showIdentity' is ambiguous
6	10	D:\Data\Awais_Laptop\University\Courses\Session\Sprin...	[Note] candidates are: void Person::showIdentity()
6	10	D:\Data\Awais_Laptop\University\Courses\Session\Sprin...	[Note] void Person::showIdentity()

Because TeachingAssistant inherits two copies of Person:

- One from Teacher
- One from Student

Compiler can't decide which one to use.

Example code with Virtual Inheritance

```
#include <iostream>
using namespace std;

class Person {
public:
    void showIdentity() {
        cout << "I am a person." << endl;
    }
};

// Use virtual inheritance
class Teacher : virtual public Person {
public:
    void teach() {
        cout << "Teaching students." << endl;
    }
};

class Student : virtual public Person {
public:
    void study() {
        cout << "Studying courses." << endl;
    }
};

class TeachingAssistant : public Teacher, public Student {
public:
    void assist() {
        cout << "Assisting in labs and grading." << endl;
    }
};
```

```
int main() {
    TeachingAssistant ta;
    ta.showIdentity(); //  No ambiguity
    ta.teach();
    ta.study();
    ta.assist();
    return 0;
}
```

Virtual Inheritance to Solve Diamond Problem

- Virtual inheritance ensures that only one copy of the base class (Person) is shared across the inheritance tree, even if multiple paths exist.
- Without virtual inheritance:
 - TeachingAssistant gets 2 copies of Person → causes ambiguity.
- With virtual inheritance:
 - TeachingAssistant gets 1 shared copy of Person → no ambiguity.

```
class Derived : virtual public Base {};
```

Task – Single Inheritance

- You are building a student information system. Create a **Person** class and a **Student** class that inherits from it.
- **Classes & Requirements:**
 - **Person:**
Attributes: name, age
Functions: display_person_info()
 - **Student** (inherits Person):
Additional Attribute: student_id
Additional Function: display_student_info()
- **Tasks for Students:**
 - Define both classes using **single inheritance**.
 - Accept input from the user for all fields.
 - Display complete student information using both functions.

Task – Multilevel Inheritance

- You are designing a payroll system.
- **Classes & Requirements:**
 - ❑ **Person:**
Attributes: name, age
Function: display_person()
 - ❑ **Employee** (inherits Person):
Additional Attribute: employee_id
Additional Function: display_employee()
 - ❑ **Manager** (inherits Employee):
Additional Attribute: department
Function: display_manager()
- **Tasks for Students:**
 - ❑ Implement **multilevel inheritance**.
 - ❑ Accept input and display complete info using all display functions.

Task – Hierarchical Inheritance

- In an organization, Employee is a base class. Developer and Designer inherit from it.
- **Classes & Requirements:**
 - ❑ **Employee:**
Attributes: name, salary
Function: display_employee()
 - ❑ **Developer** (inherits Employee):
Additional Attribute: programming_language
Additional Function: display_developer()
 - ❑ **Designer** (inherits Employee):
Additional Attribute: design_tool
Function: display_designer()
- **Tasks for Students:**
 - ❑ Implement hierarchical inheritance.
 - ❑ Create objects of both derived classes and display all information.

Task – Multiple Inheritance

- Build a device that behaves as both a Printer and a Scanner.
- **Classes & Requirements:**
 - ❑ **Printer:**
Function: print_document ()
 - ❑ **Scanner**
Function: scan_document()
 - ❑ **Photocopier** (inherits from Printer and Scanner):
Function: photocopy() (calls both above functions)

Tasks for Students:

- ❑ Implement multiple inheritance.
- ❑ Create an object of Photocopier and call all functions.

REFERENCES

- <https://www.geeksforgeeks.org/inheritance-in-c/>
- https://www.w3schools.com/cpp/cpp_inheritance.asp